

第 46 讲：BatchNorm

为什么要利用一个 Mini-Batch 的统计量，以及为什么 Training / Evaluation 行为不同？

《面向 LLM / Agent Systems 科研的深度学习零基础课》

【科研洞察】

第 45 讲解决的是“**为什么需要 Normalization**”：网络越深，Activation 的均值、方差和数值尺度越容易漂移。第 46 讲开始回答“**具体怎么做**”。Batch Normalization (BatchNorm / BN) 的核心选择非常明确：**同一个 Feature / Channel，在一个 Mini-Batch 中收集一组数，计算这组数的均值和方差，再把它们重新放到更可控的尺度上。**

这带来一个非常重要、也非常反直觉的性质：**训练时一个样本的 BatchNorm 输出，不只由它自己决定，还会受到同一个 Batch 中其他样本的影响。**这也是理解 BatchNorm、Small-Batch 问题，以及它为什么没有成为现代 LLM 主流 Normalization 的关键。

一、先把一句话讲透：BatchNorm 的“Batch”到底在哪里？

假设某一层输出一个二维 Tensor：

$$X \in \mathbb{R}^{N \times C}$$

其中：

- N ：Mini-Batch 中的样本数量；
- C ：Feature / Channel 数量；
- $x_{i,c}$ ：第 i 个样本、第 c 个 Feature 的数值。

BatchNorm 并不是把整个矩阵一次性求一个均值。它会**保持 Feature / Channel 彼此独立**：对于每一个 c ，沿 Batch 轴收集

$$[x_{1,c}, x_{2,c}, \dots, x_{N,c}]$$

然后为这一列单独计算均值与方差。

【原理】

对于二维输入 (N, C) ，BatchNorm1d 可以先这样理解：

每一列是一个 Feature；沿着 N 这一维计算统计量。

所以它不是“每个样本自己归一化自己”，而是“同一个 Feature 在一批样本之间共同确定统计量”。

二、【具体数字例子】手算一次 BatchNorm

取一个很小的 Batch：

$$X = \begin{bmatrix} 1 & 10 & 100 \\ 2 & 12 & 90 \\ 3 & 14 & 110 \\ 4 & 16 & 100 \end{bmatrix}$$

因此：

$$X.shape = (4, 3)$$

也就是 4 个样本、3 个 Feature。

先看 Feature 0：

$$[1, 2, 3, 4]$$

均值：

$$\mu_0 = \frac{1 + 2 + 3 + 4}{4} = 2.5$$

方差（此处用 BatchNorm Forward 对当前 Batch 使用的总体形式）：

$$\sigma_0^2 = \frac{(1 - 2.5)^2 + (2 - 2.5)^2 + (3 - 2.5)^2 + (4 - 2.5)^2}{4} = 1.25$$

标准化：

$$\hat{x}_{i,0} = \frac{x_{i,0} - 2.5}{\sqrt{1.25 + \epsilon}}$$

若暂时忽略极小的 ϵ ，约得到：

$$[-1.342, -0.447, 0.447, 1.342]$$

同理，三个 Feature 的 Batch Mean 为：

$$[2.5, 13, 100]$$

Batch Variance 为：

$$[1.25, 5, 50]$$

实验中，手算与 PyTorch `nn.BatchNorm1d` 的最大绝对误差只有：

$$2.38 \times 10^{-7}$$

属于浮点舍入误差。

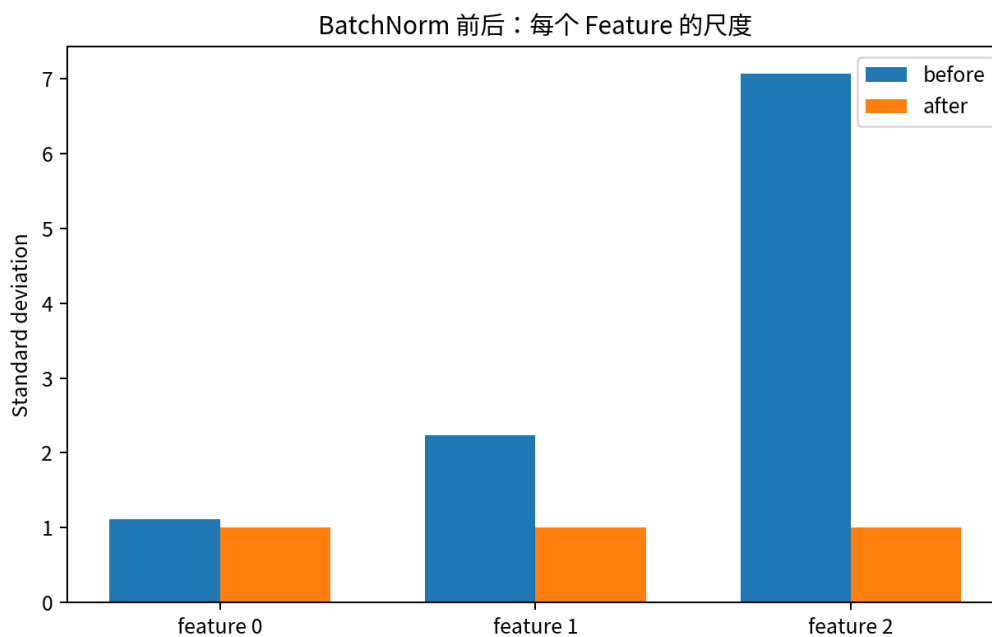


图 1: 同一个 Batch 中, 不同 Feature 原始尺度差异很大; BatchNorm 后每个 Feature 的标准差被拉回约 1。

三、BatchNorm 的完整公式：不只是“减均值除标准差”

对某个 Feature / Channel c , 训练阶段先计算:

$$\mu_{B,c} = \frac{1}{m} \sum_{i=1}^m x_{i,c}$$

$$\sigma_{B,c}^2 = \frac{1}{m} \sum_{i=1}^m (x_{i,c} - \mu_{B,c})^2$$

然后:

$$\hat{x}_{i,c} = \frac{x_{i,c} - \mu_{B,c}}{\sqrt{\sigma_{B,c}^2 + \epsilon}}$$

但真正输出不是只停在 $\hat{x}$, 而是:

$$y_{i,c} = \gamma_c \hat{x}_{i,c} + \beta_c$$

这里每个 Channel 都有自己的:

$$\gamma_c, \beta_c$$

并且它们都是**可学习 Parameter**。

【原理】

标准化先暂时把中心和尺度拉回可控区域; γ 和 β 再给网络自由度, 让它自己学习“最终到底应该用什么尺度、什么中心”。所以 BatchNorm 并不是强迫所有层永远保持均值 0、标准差 1。

例如实验设置:

$$\gamma = [2, 0.5, 1.5], \quad \beta = [1, -1, 0.2]$$

BatchNorm 输出的三个 Feature 均值约为：

$$[1, -1, 0.2]$$

标准差约为：

$$[2, 0.5, 1.5]$$

这正好说明了 γ/β 的作用。

四、为什么必须有 ϵ ?

如果某一个 Feature 在一个 Batch 中所有数都一样，例如：

$$[5, 5, 5, 5]$$

则：

$$\sigma^2 = 0$$

如果直接除以：

$$\sqrt{\sigma^2}$$

就会出现除 0。

因此实际公式写成：

$$\frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

ϵ 通常是很小的正常数。它首先是**数值稳定项**，但当方差本身也极小时，它也会实际参与尺度计算，所以不能把它理解成“理论上完全不存在”。

五、最关键的 Shape / Axis: BatchNorm 到底对哪些数一起统计 ?

模块	常见输入 Shape	对每个 Channel 统计哪些轴
BatchNorm1d	(N, C)	沿 N 统计，每个 C 独立
BatchNorm1d	(N, C, L)	沿 (N, L) 统计，每个 C 独立
BatchNorm2d	(N, C, H, W)	沿 (N, H, W) 统计，每个 C 独立
BatchNorm3d	(N, C, D, H, W)	沿 (N, D, H, W) 统计，每个 C 独立

因此在 CNN 中，BatchNorm2d 的思想是：**同一个 Channel 在整个 Mini-Batch 和空间位置上共同估计统计量。**

本讲实验构造：

$$X.shape = (2, 2, 2, 2)$$

手工沿 (N, H, W) 计算两个 Channel 的均值：

$$[4.5, 45]$$

方差：

$$[5.25, 525]$$

手工结果与 BatchNorm2d 最大误差约:

$$1.19 \times 10^{-7}$$

BatchNorm2d 的统计轴

Input shape: (N, C, H, W)

For each channel C independently:

collect values across N, H, W

compute mean / variance -> normalize -> gamma / beta

Statistics axes = (N, H, W); channel axis C is kept separate.

图 2: BatchNorm2d 保留 Channel 轴，为每个 Channel 沿 N, H, W 共同估计统计量。

六、Training 时到底发生什么？

调用：

```
model.train()
y = batch_norm(x)
```

BatchNorm 会做两件不同的事：

1. 用当前 Mini-Batch 的 mean / variance 归一化当前输入；
2. 更新 running_mean / running_var，供以后 Evaluation 使用。

这是 BatchNorm 与很多普通层非常不同的地方：它不仅有可学习的 γ/β ，还维护运行统计状态 (running statistics)。

PyTorch 中常见状态包括：

- weight: γ ;
- bias: β ;
- running_mean;
- running_var;
- num_batches_tracked。

其中 running mean 的更新可直观写成：

$$\mu_{run} \leftarrow (1 - \alpha)\mu_{run} + \alpha\mu_B$$

这里 PyTorch 的 BatchNorm 参数 `momentum=0.1` 中的 0.1 就扮演 α 。

【常见误区 / 风险】

BatchNorm 的 `momentum` 和第 37 讲 Optimizer Momentum 不是同一个概念。两者名字相同，但状态、对象和公式语义不同。看到代码时必须先确认：这是 `BatchNorm(momentum=...)`，还是 `SGD(momentum=...)`。

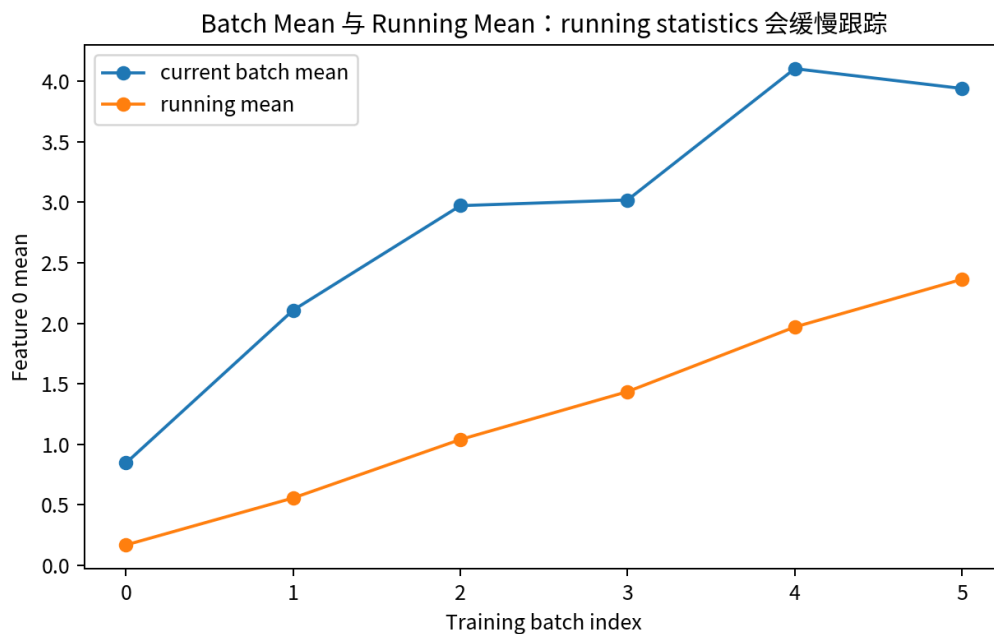


图 3: 当前 Batch Mean 会明显波动；running mean 以移动平均方式缓慢跟踪训练数据统计。

一个 PyTorch 实现细节: Batch Variance 与 Running Variance 的估计形式

在本课程当前 PyTorch 环境中，训练 Forward 对当前 Batch 归一化时，使用相当于：

`var(X, unbiased=False)`

的方差；而更新 `running_var` 时使用无偏形式的方差估计。对于小 Batch，两者可能有可见差别。

这不是理解 BatchNorm 核心机制时必须死记的第一优先级，但如果你以后自己手算并逐元素对齐 PyTorch，这个细节很重要。

七、Evaluation 为什么不能继续使用当前 Batch 的统计量？

推理阶段可能出现：

- 一次只输入 1 个样本；
- Batch Size 与训练时完全不同；
- Batch 中样本组成随机变化；
- 在线服务中请求一个一个到达。

如果 Evaluation 仍然依赖“当前 Batch”，那么同一个样本仅仅因为旁边换了几个样本，输出就可能发生变化。

所以当：

```
model.eval()
```

BatchNorm 通常改用训练期间累积的：

```
running_mean, running_var
```

评估公式近似为：

$$y = \gamma \frac{x - \mu_{run}}{\sqrt{\sigma_{run}^2 + \epsilon}} + \beta$$

本讲实验中最终 running mean 为：

```
[2.363, 2.321, 2.387]
```

固定 probe：

```
[1, 2, 3]
```

评估输出：

```
[-0.852, -0.220, 0.404]
```

手工使用 running statistics 计算，与 PyTorch 最大误差只有 5.96×10^{-8} 。

八、一个反直觉事实：同一个样本，Training 输出会因为“同伴”不同而改变

固定样本：

```
x = [1, 1, 1]
```

当它和接近 0 的样本组成一个 Batch 时，第一个 Feature 的归一化输出约：

```
1.342
```

当它和接近 10 的样本组成另一个 Batch 时，同一个数 1 的输出却变成：

```
-1.704
```

两者差异：

```
3.046
```

原因不是样本自己变了，而是：

$$\mu_B, \sigma_B^2$$

变了。

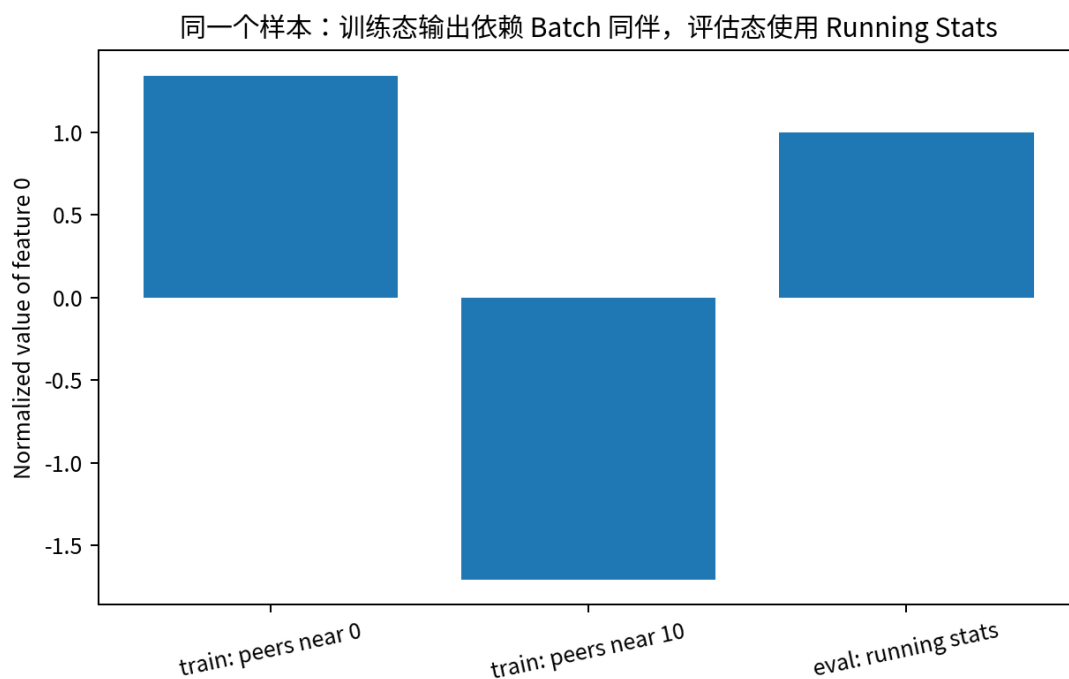


图 4: Training 模式下，同一个样本依赖 Batch 同伴；Eval 模式使用固定 running statistics，输出稳定。

【原理】

这就是 BatchNorm 的核心 batch coupling：训练时每个样本不是完全独立地经过 BN。它的归一化结果依赖同一统计组中的其他数。

九、Batch Size 为什么会影响 BatchNorm ?

BatchNorm 依赖样本统计量去估计总体统计量。Batch 很小时，均值和方差估计自然更不稳定。

本讲从总体：

$$X \sim \mathcal{N}(3, 2^2)$$

重复抽取 2000 个 Batch。得到：

Batch Size	std(batch mean)	std(batch variance)
2	1.398	3.004
4	1.021	2.351
8	0.694	1.873
16	0.488	1.361
32	0.360	0.981
64	0.252	0.692
128	0.176	0.517

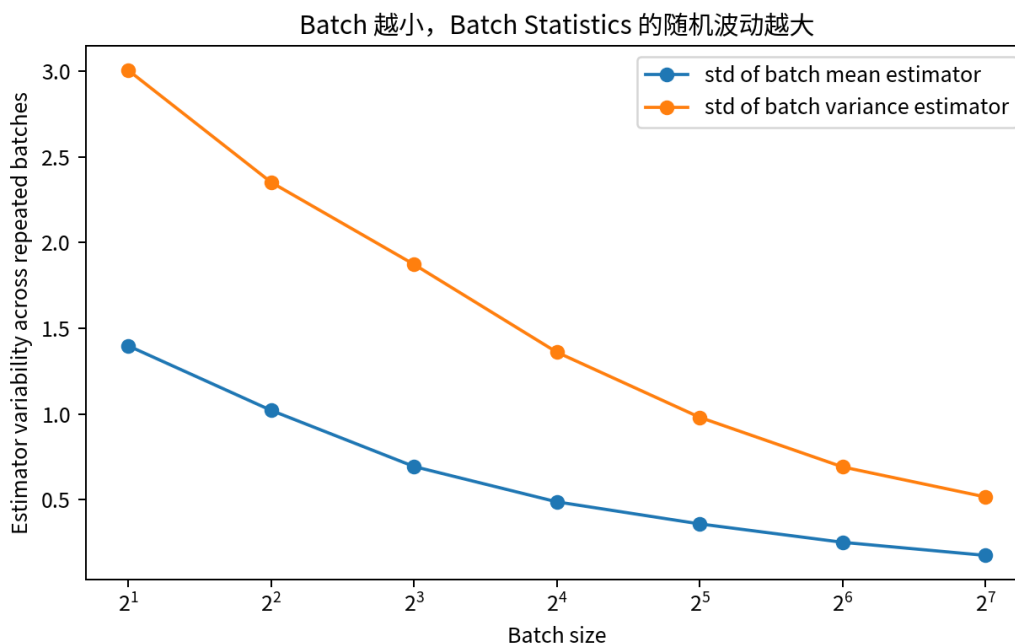


图 5: Batch 越小，同一总体分布反复抽样得到的 Batch Mean / Variance 越不稳定。

这意味着 Small-Batch 下 BN 可能出现：

- 归一化噪声更大；
- running statistics 更难稳定；
- 不同 Batch 组合造成更强输出波动；
- 训练与 Evaluation 的统计差异可能更明显。

Batch Size = 1 一定不能用 BatchNorm 吗？

不能只看 N 。关键是：每个 Channel 用来估计统计量的有效元素数有多少。

实验中：

$$(N, C) = (1, 3)$$

训练态会报错：每个 Channel 只有 1 个值，无法正常估计训练所需统计量。

但：

$$(N, C, L) = (1, 3, 4)$$

仍可以运行，因为每个 Channel 实际有 $N \times L = 4$ 个值参与统计。

十、BatchNorm 的 Backward：为什么 Gradient 也会跨样本耦合？

这是理解即可，但非常重要。

Forward 中：

$$\mu_B, \sigma_B^2$$

由同一组样本共同计算，所以改变任意一个 x_i ，不仅会直接改变自己的 $\hat{x}_i$ ，还会改变共享的均值、方差，从而间接影响其他样本的归一化值。

因此 Backward 时:

$$\frac{\partial L}{\partial x_i}$$

并不只由第 i 个样本自己的 upstream gradient 决定。

一个常见的紧凑形式是:

$$\frac{\partial L}{\partial x_i} = \frac{\gamma}{m\sqrt{\sigma_B^2 + \epsilon}} \left[md_i - \sum_j d_j - \hat{x}_i \sum_j d_j \hat{x}_j \right]$$

其中 $d_i = \partial L / \partial y_i$ 。

你当前不需要背这个公式，只需要理解：

BN Forward 共享统计量 $\Rightarrow$ BN Backward 也会在统计组内产生耦合。

十一、BatchNorm 是 Regularization 吗？

它的首要设计目标是 Normalization / Optimization Stability，而不是像 Weight Decay 那样明确给目标函数增加惩罚，也不是像 Dropout 那样显式随机屏蔽激活。

但是训练阶段使用 Mini-Batch 统计量会带来随机性：同一个样本因为 Batch 组成不同，归一化结果有轻微变化。因此 BatchNorm 可能产生一定的**隐式正则化效应**。

【常见误区 / 风险】

不要把 “BatchNorm 有时带来 regularization-like effect” 简化成 “BatchNorm 就等于 Regularization”。它同时改变优化几何、信号尺度和 batch coupling，作用机制比单一正则项复杂得多。

十二、BatchNorm 通常放在 Activation 前还是后？

经典 CNN 代码经常写成：

Conv / Linear $\rightarrow$ BatchNorm $\rightarrow$ ReLU

例如：

```
self.block = nn.Sequential(
    nn.Linear(128, 256),
    nn.BatchNorm1d(256),
    nn.ReLU(),
)
```

直觉是先让 Linear / Conv 产生 pre-activation，再控制其统计尺度，然后进入 Activation。

但 “BN 必须永远放在 ReLU 前” 不是普适数学定律。具体架构可以有不同设计。读论文或代码时，应记录真实顺序，而不是根据记忆自动脑补。

十三、PyTorch 中最小可运行例子

```
import torch
```

```
import torch.nn as nn

x = torch.randn(32, 128) # (N, C)
bn = nn.BatchNorm1d(128) # 每个 Channel 有 gamma / beta

bn.train() # 使用当前 batch 统计量
train_y = bn(x)

bn.eval() # 使用 running mean / running var
with torch.no_grad():
    eval_y = bn(x)
```

一个 BatchNorm1d(128) 的常见状态 Shape:

- `weight.shape = (128,)`;
- `bias.shape = (128,)`;
- `running_mean.shape = (128,)`;
- `running_var.shape = (128,)`。

前两者是 Parameter; running statistics 通常作为 Buffer / persistent state 保存, 不由 Optimizer 像普通 Parameter 那样更新。

十四、分布式训练时又多了一层问题：谁和谁组成 “Batch”？

假设 Global Batch Size = 256, 但分布在 8 张 GPU 上, 每张 GPU 只有 32 个样本。如果普通 BatchNorm 每个设备各算各的统计量, 那么每个 BN 实际看到的是 local batch 32, 而不是 global batch 256。

这会带来:

- 不同设备的 Batch Statistics 不同;
- local batch 太小时统计更噪;
- 训练行为与单卡大 Batch 不完全等价。

一些视觉训练会使用 SyncBatchNorm 在设备之间同步统计量, 但这又会增加通信成本。

所以以后论文写 “Batch Size = 256” 时, 若模型高度依赖 BatchNorm, 还需要追问:

BN statistics 是 per-device 还是 synchronized?

十五、为什么 CNN 常见 BatchNorm, 而现代 Transformer / LLM 更常见 LayerNorm / RMSNorm ?

BatchNorm 的统计天然依赖 Batch / Channel 组织方式, 而 Transformer 的典型隐藏状态是:

$$X.shape = (B, T, D)$$

对于语言模型, 存在几个现实问题:

- Batch Size 可能因为显存、Sequence Length 而频繁改变；
- 不同 Sequence 有 Padding、有效 Token 数不同；
- 自回归推理可能一次只生成一个 Token / 一个请求；
- 分布式训练中 micro-batch 可能很小；
- 希望单个 Token 的归一化尽量不依赖“同 Batch 里来了哪些别的请求”。

LayerNorm / RMSNorm 更自然地沿隐藏维 D 对每个 Token 自己做 Normalization，因此不需要依赖 Batch 同伴。

【与 LLM / Transformer / Agent Systems 的关系】

现代 Transformer / LLM 中，你更经常看到：

$$(B, T, D) \xrightarrow{\text{LayerNorm/RMSNorm}} (B, T, D)$$

统计发生在每个 Token 自己的 Hidden Dimension 上。下一讲会正式把 BatchNorm、LayerNorm、RMSNorm 放在同一张 Axis 图里比较。

对 Agent Systems 来说，这一点也有现实意义：Agent rollout / trajectory 的 Batch 往往大小不固定，甚至在线运行时一次只有一个状态。依赖 Batch 统计的模块会让行为与 batch composition 绑定得更紧。

十六、科研实验中怎样诊断 BatchNorm 问题？

如果训练异常，可以按下面顺序检查：

1. 输入 Shape 是否符合模块预期？例如 BatchNorm1d(C) 的 Channel 是否真的是第 2 维？
2. 实际每个 Channel 有多少元素参与统计？local batch 是否过小？
3. Validation / Test 是否忘记 `model.eval()`？
4. running mean / var 是否已经被训练数据充分更新？
5. 训练和部署数据分布是否明显变化？
6. 分布式训练时 BN 是 per-device 还是 synchronized？
7. checkpoint 是否正确保存并恢复了 running statistics？
8. 对照实验是否同时改变了 Batch Size，以至于 BN 本身的统计噪声也发生变化？

特别是做 Ablation 时，如果你把 Batch Size 从 32 改到 4，又同时说“只比较某个模块”，那么对含 BatchNorm 的网络来说，实际上你还改变了 BN 的统计质量。

十七、【常见误区】集中纠正

【常见误区 / 风险】

1. **误区：** BatchNorm 是把整个 Batch 所有数一起变成 $\text{mean}=0$ 、 $\text{std}=1$ 。
纠正： 通常是每个 Channel 独立统计；必须看 Shape 和 Axis。
2. **误区：** `model.eval()` 等于 `torch.no_grad()`。
纠正： 前者改变 BatchNorm/Dropout 等模块行为，后者控制 Autograd。
3. **误区：** running mean / var 是 Optimizer 学出来的 Parameter。
纠正： 它们是运行统计状态，不是普通 Gradient Parameter。
4. **误区：** Batch 越大一定越好。
纠正： 大 Batch 的统计更稳定，但训练效率、显存、优化噪声等还有其他权衡。
5. **误区：** 有 BatchNorm 就不需要好的 Initialization。
纠正： BN 可以缓解尺度问题，但不意味着其他稳定性设计全部失效。

十八、本讲最低掌握标准

学完本讲，你至少应该能独立回答：

1. 对 (N, C) 输入，BatchNorm1d 的 mean / variance 沿哪个轴计算？
2. 为什么 BatchNorm 后还需要可学习的 γ, β ？
3. 为什么训练态一个样本会依赖同 Batch 里的其他样本？
4. 为什么 `train()` 和 `eval()` 下 BatchNorm 行为不同？
5. running mean / running var 是做什么的？
6. 为什么 Small Batch 会让 BatchNorm 更不稳定？
7. BatchNorm2d 对 (N, C, H, W) 到底在哪些轴上统计？
8. 为什么 CNN 常见 BatchNorm，而现代 LLM 更常见 LayerNorm / RMSNorm？

【原理】

本讲最值得留下的一句话是：

BatchNorm = 每个 Channel 利用一组 Batch Statistics 做标准化，再由 γ/β 恢复可学习尺度。

而它最独特的代价是：

Training 时样本之间通过 Batch Statistics 发生耦合。

十九、BatchNorm 的 `state_dict`：为什么 checkpoint 不只是保存 Weight？

以：

```
bn = nn.BatchNorm1d(128)
print(bn.state_dict().keys())
```

为例，常见条目不只有：

`weight, bias`

还会包括：

`running_mean, running_var, num_batches_tracked`

这意味着一个训练好的 BatchNorm 模型，其推理行为依赖的不只是可学习 Parameter，也依赖训练期间积累的运行统计状态。

如果只复制 Linear / Conv 的 Weight，却没有正确恢复 BatchNorm 的 running statistics，那么即使所有可学习参数完全相同：

Evaluation 输出仍可能不同。

因此 checkpoint：

```
torch.save(model.state_dict(), "model.pt")
```

通常会把这些 Buffer 一起保存。

【科研洞察】

这也是为什么科研复现不能只说“我拿到了模型权重”。对于存在 BatchNorm 的架构，还要确认 state dict 是否完整、模型是否处于正确的 train/eval 状态，以及 running statistics 是否来自同一个训练过程。

二十、完整执行轨迹：Linear $\rightarrow$ BatchNorm $\rightarrow$ ReLU

考虑：

```
block = nn.Sequential(
    nn.Linear(4, 8),
    nn.BatchNorm1d(8),
    nn.ReLU(),
)
```

若输入：

$X.shape = (32, 4)$

第一步 Linear：

$(32, 4) \rightarrow (32, 8)$

它为 32 个样本各自产生 8 个 pre-activation。

第二步 BatchNorm：对 8 个输出 Feature 分别沿 32 个样本统计：

$\mu_B.shape = (8,), \quad \sigma_B^2.shape = (8,)$

然后用同一组 $\mu_{B,c}, \sigma_{B,c}$ 处理这一 Batch 内 Feature c 的所有 32 个值，因此：

$(32, 8) \rightarrow (32, 8)$

Shape 不变。

第三步 ReLU 逐元素执行：

$$(32, 8) \rightarrow (32, 8)$$

训练态的完整逻辑是：

$$X \rightarrow \text{Linear} \rightarrow \underbrace{\text{current batch stats}}_{BN} \rightarrow \text{ReLU}$$

并同时更新 running statistics。

评估态则变成：

$$X \rightarrow \text{Linear} \rightarrow \underbrace{\text{running stats}}_{BN} \rightarrow \text{ReLU}$$

【常见误区 / 风险】

如果 Validation 时忘记调用 `model.eval()`，你的验证指标不仅是在评估模型参数，还混入了“当前 Validation Batch 是怎样分组的”这一额外随机因素。对含 BatchNorm 的模型，这可能让评估协议发生实质变化。

二十一、先预览一张表：BatchNorm / LayerNorm / RMSNorm 的根本差别

下一讲会详细推导，这里先只建立 Axis 直觉：

方法	统计量主要来自哪里	是否依赖 Batch 同伴
BatchNorm	同一 Channel 在 Batch（以及可能的空间维）中的一组数	是
LayerNorm	单个样本 / Token 自己的 Hidden Dimension	否
RMSNorm	单个样本 / Token 自己的 Hidden Dimension 的 RMS	否

对于 Transformer 隐藏状态：

$$X.shape = (B, T, D)$$

现代 LLM 常希望第 (b, t) 个 Token 可以独立用自己的 D 维特征做尺度控制，而不是依赖同 Batch 里其他序列。因此 LayerNorm / RMSNorm 与 Transformer 的数据组织方式更自然匹配。

这一点不是说 BatchNorm “不好”，而是：

Normalization 方法是否合适，取决于你想让哪些元素共享统计量。

二十二、下一讲预告

第 47 讲将进入：

LayerNorm 与 RMSNorm

并重点比较：

- BatchNorm: 跨 Batch / Channel 组织统计;
- LayerNorm: 对单个样本 / Token 的 Hidden Dimension 统计;
- RMSNorm: 不减均值, 只用 RMS 控制尺度;
- 为什么 Transformer Block 中通常采用 LayerNorm / RMSNorm;
- Pre-Norm / Post-Norm 怎样影响深层 Transformer 的 Gradient Flow。

课后 5 分钟口述自检

不看讲义, 尝试连续说清下面这条链:

$$(N, C) \rightarrow \mu_B, \sigma_B^2 \rightarrow \hat{X} \rightarrow \gamma \hat{X} + \beta \rightarrow \text{running stats} \rightarrow \text{train/eval}$$

然后再回答三个“为什么”:

1. 为什么同一个样本在 train 模式下会随 Batch 同伴变化?
2. 为什么 Small Batch 会让这种变化更明显?
3. 为什么这两个性质会让 Transformer / LLM 更偏向 LayerNorm / RMSNorm?

最小代码记忆

```
bn = nn.BatchNorm1d(C)

model.train() # 当前 batch stats + 更新 running stats
train_y = bn(x)

model.eval() # 使用 running stats
with torch.no_grad():
    eval_y = bn(x)
```

【科研洞察】

以后看到论文或代码里的 BatchNorm, 你至少应该马上检查: 输入 Shape、统计 Axis、有效 Batch 大小、train/eval 状态、running statistics、是否分布式同步。